

รายงานโครงงาน

เรื่อง ระบบตรวจวัดแอลกอฮอล์ด้วยเทคโนโลยี IoT

เสนอ

อาจารย์ประเสริฐศักดิ์ อู่อรุญ

จัดทำโดย

นายณันธภัทร์ ลิ่มเจริญ รหัสนิสิต 6621653751 เลขที่ 23

นายณัฐภัทร ยืนยง รหัสนิสิต 6621655231 เลขที่ 35

นายบรรณวัฒน์ จันทร์สมมิตร รหัสนิสิต 6621655274 เลขที่ 38

รายงานนี้เป็นส่วนหนึ่งของรายวิชา 02739496

สาขาวิชาเทคโนโลยีสารสนเทศ คณะศิลปศาสตร์และวิทยาศาสตร์

มหาวิทยาลัยเกษตรศาสตร์ วิทยาเขตกำแพงแสน

ภาคเรียนที่ 2 ปีการศึกษา 2568

คำนำ

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา 02739496 จัดทำขึ้นเพื่อนำเสนอโครงงานระบบตรวจวัดแอลกอฮอล์ด้วยเทคโนโลยี IoT ซึ่งใช้บอร์ด ESP32 ร่วมกับเซนเซอร์ MQ-3 ตรวจวัดไอแอลกอฮอล์ในลมหายใจ แสดงผลบนจอ OLED และไฟ LED ส่งข้อมูลผ่านแพลตฟอร์ม NETPIE ด้วยโปรโตคอล MQTT และจัดเก็บประวัติการตรวจวัดบน Firebase โดยมีแอปพลิเคชันที่พัฒนาด้วย Flutter สำหรับติดตามผลและบันทึกข้อมูลผู้เป่า

เนื้อหาของรายงานแบ่งออกเป็น 5 บท ได้แก่ บทนำ แนวคิดและทฤษฎีที่เกี่ยวข้อง การออกแบบและพัฒนาระบบ ผลการดำเนินงาน และสรุปผลและข้อเสนอแนะ โดยอธิบายการทำงานของระบบอ้างอิงจากซอร์สโค้ดจริงที่เผยแพร่บน GitHub พร้อมแผนภาพสถาปัตยกรรม ผังงาน และภาพการทำงานของอุปกรณ์และแอปพลิเคชัน รวมทั้งแนบซอร์สโค้ดเฟิร์มแวร์ไว้ในภาคผนวก

คณะผู้จัดทำขอขอบพระคุณอาจารย์ประเสริฐศักดิ์ อู่อรุญ ที่ให้ความรู้และคำแนะนำตลอดการจัดทำโครงงาน หวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์แก่ผู้ที่สนใจการประยุกต์ใช้เทคโนโลยี IoT ในงานด้านความปลอดภัย หากมีข้อผิดพลาดประการใด คณะผู้จัดทำขออภัยมา ณ ที่นี

คณะผู้จัดทำ

สารบัญ

หน้า

คำนำ ..... ก
สารบัญ ..... ข
สารบัญตาราง ..... ง
สารบัญภาพ ..... จ
บทที่ 1 บทนำ 1
1.1 ที่มาและความสำคัญ 1
1.2 วัตถุประสงค์ 1
1.3 ขอบเขตของโครงงาน 1
1.4 ประโยชน์ที่คาดว่าจะได้รับ 2
1.5 นิยามศัพท์เฉพาะ 3
บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง 4
2.1 อินเทอร์เน็ตของสรรพสิ่ง (Internet of Things) 4
2.2 ปริมาณแอลกอฮอล์ในเลือดและเกณฑ์ตามกฎหมาย 4
2.3 เซนเซอร์ตรวจวัดแอลกอฮอล์ MQ-3 4
2.4 บอร์ดไมโครคอนโทรลเลอร์ ESP32 5
2.5 จอแสดงผล OLED และการสื่อสารแบบ I2C 5
2.6 โปรโตคอล MQTT และแพลตฟอร์ม NETPIE 5
2.7 Firebase 6
2.8 Flutter และการจัดการสถานะด้วย Provider 6

สารบัญ

หน้า

บทที่ 3 การออกแบบและพัฒนาระบบ..... 7

3.1 สถาปัตยกรรมของระบบ..... 7

3.2 อุปกรณ์ที่ใช้ในการพัฒนา..... 8

3.3 การต่อวงจร..... 9

3.4 เครื่องมือและไลบรารีที่ใช้ในการพัฒนา..... 9

3.5 การพัฒนาเฟิร์มแวร์..... 11

3.6 การสื่อสารข้อมูลผ่าน MQTT..... 13

3.7 การออกแบบการจัดเก็บข้อมูล..... 14

3.8 การพัฒนาแอปพลิเคชัน..... 16

3.9 ขั้นตอนการพัฒนา..... 17

บทที่ 4 ผลการดำเนินงาน..... 19

4.1 ต้นแบบอุปกรณ์..... 19

4.2 แอปพลิเคชัน..... 19

4.3 ข้อมูลบน Cloud Firestore..... 21

4.4 ผลการทดสอบ..... 21

4.5 ภาพการดำเนินงาน..... 22

บทที่ 5 สรุปผล อภิปรายผล และข้อเสนอแนะ..... 23

5.1 สรุปผล..... 23

5.2 อภิปรายผล..... 23

5.3 ปัญหาและข้อจำกัด..... 23

5.4 ข้อเสนอแนะ..... 24

บรรณานุกรม..... 25

ภาคผนวก ก ที่เก็บซอร์สโค้ดและวิธีติดตั้ง..... 27

ภาคผนวก ข ซอร์สโค้ดเฟิร์มแวร์..... 28

สารบัญตาราง

ตาราง **หน้า**

ตารางที่ 1.1 นิยามศัพท์เฉพาะ....3
ตารางที่ 2.1 เกณฑ์ปริมาณแอลกอฮอล์ในเลือดของผู้ขับขี่....4
ตารางที่ 2.2 คุณสมบัติของ ESP32 ที่เกี่ยวข้องกับโครงงาน....5
ตารางที่ 3.1 อุปกรณ์ที่ใช้ในการพัฒนา....8
ตารางที่ 3.2 การเชื่อมต่ออุปกรณ์กับขาของบอร์ด ESP32....9
ตารางที่ 3.3 เครื่องมือและไลบรารีที่ใช้ในการพัฒนา....10
ตารางที่ 3.4 ระดับการแจ้งเตือน....12
ตารางที่ 3.5 ค่าคงที่สำคัญในเฟิร์มแวร์....13
ตารางที่ 3.6 หัวข้อ MQTT ที่ใช้ในระบบ....14
ตารางที่ 3.7 โครงสร้างเอกสารในคอลเลกชัน measurements....15
ตารางที่ 3.8 โครงสร้างซอร์สโค้ดของแอปพลิเคชัน....16
ตารางที่ 3.9 เงื่อนไขการตรวจสอบข้อมูลในแบบฟอร์ม....17
ตารางที่ 4.1 ผลการทดสอบการทำงานของระบบ....21
ตารางที่ 4.2 ผลการตรวจสอบการคอมไพล์และคุณภาพของซอร์สโค้ด....22

สารบัญภาพ

ภาพ **หน้า**

ภาพที่ 3.1 สถาปัตยกรรมของระบบตรวจวัดแอลกอฮอล์..... 7
ภาพที่ 3.2 ผังงานการทำงานของเฟิร์มแวร์บน ESP32..... 11
ภาพที่ 4.1 ต้นแบบอุปกรณ์ตรวจวัดแอลกอฮอล์..... 19
ภาพที่ 4.2 หน้าวัดค่าและหน้าประวัติของแอปพลิเคชัน..... 20
ภาพที่ 4.3 หน้าตั้งค่าของแอปพลิเคชัน..... 20
ภาพที่ 4.4 ข้อมูลการตรวจวัดที่บันทึกบน Cloud Firestore..... 21
ภาพที่ 4.5 ภาพการดำเนินงานของคณะผู้จัดทำ..... 22

บทที่ 1
บทนำ

1.1 ที่มาและความสำคัญ

ปัญหาการดื่มเครื่องดื่มแอลกอฮอล์แล้วขับขี่ยังคงเป็นสาเหตุสำคัญของอุบัติเหตุบนท้องถนน กฎหมายไทยจึงกำหนดเกณฑ์ปริมาณแอลกอฮอล์ในเลือดของผู้ขับขี่ไว้อย่างชัดเจน โดยผู้ขับขี่ทั่วไปต้องมีปริมาณแอลกอฮอล์ในเลือดไม่เกิน 50 มิลลิกรัมเปอร์เซ็นต์ และผู้ขับขี่บางกลุ่ม เช่น ผู้ที่อายุต่ำกว่า 20 ปี หรือผู้ที่มีใบอนุญาตขับรถชั่วคราว ต้องไม่เกิน 20 มิลลิกรัมเปอร์เซ็นต์ (กฎกระทรวง ฉบับที่ 21, 2560) การตรวจวัดเบื้องต้นที่ทำได้รวดเร็วจึงช่วยคัดกรองและป้องกันความเสี่ยงก่อนการขับขี่

คณะผู้จัดทำจึงพัฒนาระบบตรวจวัดแอลกอฮอล์ในลมหายใจด้วยเทคโนโลยีอินเทอร์เน็ตของสรรพสิ่ง (Internet of Things: IoT) ประกอบด้วยอุปกรณ์ที่ใช้บอร์ด ESP32 ร่วมกับเซนเซอร์ MQ-3 ซึ่งแสดงผลทันทีบนจอ OLED และไฟ LED ส่งข้อมูลผ่านแพลตฟอร์ม NETPIE ด้วยโปรโตคอล MQTT และแอปพลิเคชันบนสมาร์ตโฟนที่พัฒนาด้วย Flutter สำหรับติดตามค่าแบบเรียลไทม์ บันทึกข้อมูลผู้เข้ารับการตรวจ และปรับเกณฑ์การแจ้งเตือนจากระยะไกล โดยจัดเก็บข้อมูลบน Firebase เพื่อใช้ตรวจสอบย้อนหลัง ระบบนี้สามารถประยุกต์ใช้ในสถานศึกษา องค์กร หรือจุดคัดกรองเบื้องต้น

1.2 วัตถุประสงค์

1. เพื่อพัฒนาระบบตรวจวัดระดับแอลกอฮอล์ในลมหายใจโดยใช้เทคโนโลยี IoT
2. เพื่อแสดงผลค่าที่ตรวจวัดได้แบบเรียลไทม์ทั้งบนจอแสดงผลของอุปกรณ์และบนแอปพลิเคชัน
3. เพื่อจัดเก็บข้อมูลการตรวจวัดลงในฐานข้อมูลบน Firebase
4. เพื่อศึกษาและประยุกต์ใช้บอร์ดไมโครคอนโทรลเลอร์ ESP32 ร่วมกับแพลตฟอร์ม IoT
5. เพื่อเพิ่มแนวทางในการป้องกันอุบัติเหตุที่เกิดจากการดื่มแอลกอฮอล์แล้วขับขี่

1.3 ขอบเขตของโครงงาน

1. ด้านฮาร์ดแวร์ ใช้บอร์ด ESP32 DOIT DevKit V1 เซนเซอร์ MQ-3 จอ OLED ขนาด 128×64 จุด แบบ I2C และไฟ LED 2 ดวงสำหรับแจ้งเตือน
2. ด้านเฟิร์มแวร์ พัฒนาด้วยภาษา C++ บน Arduino Framework ผ่าน PlatformIO ทำหน้าที่อ่านค่าเซนเซอร์ ประมวลผล แสดงผล และส่งข้อมูลขึ้นคลาวด์
3. ด้านการสื่อสาร ใช้แพลตฟอร์ม NETPIE เป็นตัวกลางรับส่งข้อมูลผ่าน MQTT และ REST API

4. ด้านแอปพลิเคชัน พัฒนาด้วย Flutter ภาษา Dart มี 3 หน้าจอหลัก ได้แก่ หน้าวัดค่า หน้าประวัติ และหน้าตั้งค่า
5. ด้านการจัดเก็บข้อมูล ใช้ Firebase Realtime Database สำหรับค่าที่อุปกรณ์วัดได้ และ Cloud Firestore สำหรับประวัติผู้เข้ารับการตรวจ
6. ระบบเป็นต้นแบบเพื่อการศึกษา ค่าที่วัดได้เป็นค่าดัชนีในสเกล 0–100 ซึ่งยังไม่ได้สอบเทียบกับเครื่องวัดมาตรฐาน จึงไม่สามารถใช้แทนเครื่องตรวจวัดที่รับรองตามกฎหมาย

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ช่วยลดความเสี่ยงในการเกิดอุบัติเหตุบนท้องถนนจากการเมาแล้วขับ
2. ผู้ใช้งานสามารถตรวจสอบระดับแอลกอฮอล์เบื้องต้นของตนเองได้
3. มีฐานข้อมูลบน Firebase ที่บันทึกประวัติการตรวจวัด เพื่อนำไปวิเคราะห์หรือใช้งานต่อได้
4. คณะผู้จัดทำได้พัฒนาทักษะการเขียนโปรแกรมควบคุม ESP32 การพัฒนาแอปพลิเคชันด้วย Flutter และการประยุกต์ใช้เทคโนโลยี IoT

1.5 นิยามศัพท์เฉพาะ

ตารางที่ 1.1 นิยามศัพท์เฉพาะ

คำศัพท์	ความหมาย
IoT (Internet of Things)	การเชื่อมต่ออุปกรณ์ทางกายภาพเข้ากับเครือข่ายอินเทอร์เน็ต เพื่อรับส่งข้อมูลและควบคุมการทำงานได้
ไมโครคอนโทรลเลอร์	คอมพิวเตอร์ขนาดเล็กบนชิปเดียวที่ใช้ควบคุมอุปกรณ์อิเล็กทรอนิกส์ ในโครงงานนี้คือ ESP32
ADC (Analog-to-Digital Converter)	วงจรแปลงสัญญาณแอนะล็อก เช่น แรงดันจากเซนเซอร์ ให้เป็นตัวเลขดิจิทัล
MQTT	โปรโตคอลรับส่งข้อความแบบ Publish/Subscribe ที่มีขนาดเล็ก เหมาะกับอุปกรณ์ IoT
NETPIE	แพลตฟอร์ม IoT ที่ให้บริการ MQTT Broker, Device Shadow และ REST API
Device Shadow	ข้อมูลสถานะล่าสุดของอุปกรณ์ที่แพลตฟอร์มเก็บไว้ ให้แอปพลิเคชันอ่านได้แม้อุปกรณ์ไม่ได้เชื่อมต่ออยู่
Threshold	ค่าเกณฑ์ที่ใช้แบ่งระดับการแจ้งเตือน ในระบบนี้มี 2 ค่า
BAC (Blood Alcohol Concentration)	ปริมาณแอลกอฮอล์ในเลือด ในระบบนี้ใช้เรียกค่าดัชนี 0–100 ที่แปลงจากเซนเซอร์ ซึ่งไม่ใช่ค่า BAC ตามกฎหมาย

บทที่ 2
ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

2.1 อินเทอร์เน็ตของสรรพสิ่ง (Internet of Things)

IoT คือโครงสร้างพื้นฐานที่เชื่อมต่อสิ่งต่าง ๆ ทั้งทางกายภาพและทางดิจิทัลเข้าด้วยกันผ่านเทคโนโลยีสารสนเทศและการสื่อสาร (International Telecommunication Union [ITU], 2012) ระบบ IoT โดยทั่วไปประกอบด้วย 3 ส่วน ได้แก่ อุปกรณ์ที่ตรวจวัดหรือควบคุมสิ่งแวดล้อม เครือข่ายและแพลตฟอร์มที่รับส่งและจัดเก็บข้อมูล และแอปพลิเคชันที่นำข้อมูลไปแสดงผลหรือใช้ตัดสินใจ ซึ่งโครงงานนี้ออกแบบตามโครงสร้างดังกล่าว

2.2 ปริมาณแอลกอฮอล์ในเลือดและเกณฑ์ตามกฎหมาย

กฎกระทรวง ฉบับที่ 21 (พ.ศ. 2560) ออกตามความในพระราชบัญญัติจราจรทางบก พ.ศ. 2522 กำหนดเกณฑ์ที่ถือว่าผู้ขับขี่เมาสุรา ดังตารางที่ 2.1 (กฎกระทรวง ฉบับที่ 21, 2560; มติชนออนไลน์, 2560)

ตารางที่ 2.1 เกณฑ์ปริมาณแอลกอฮอล์ในเลือดของผู้ขับขี่

กลุ่มผู้ขับขี่	ถือว่าเมาสุราเมื่อเกิน
ผู้ขับขี่ที่อายุต่ำกว่า 20 ปีบริบูรณ์ ผู้ที่มีใบอนุญาตขับรถชั่วคราว ผู้ที่ไม่มีใบอนุญาตขับขี่ หรือถูกพักใช้หรือเพิกถอนใบอนุญาต	20 มิลลิกรัมเปอร์เซ็นต์
ผู้ขับขี่ทั่วไป	50 มิลลิกรัมเปอร์เซ็นต์

เกณฑ์ดังกล่าวเป็นเหตุผลที่ระบบในโครงงานนี้ออกแบบให้มีค่า Threshold 2 ระดับ และให้ผู้ใช้ปรับค่าได้จากแอปพลิเคชัน

2.3 เซนเซอร์ตรวจวัดแอลกอฮอล์ MQ-3

MQ-3 เป็นเซนเซอร์ก๊าซชนิดสารกึ่งตัวนำที่ใช้ดีบุกออกไซด์ (SnO_2) เป็นวัสดุตรวจวัด เมื่อมีไอแอลกอฮอล์ ค่าความต้านทานของเซนเซอร์จะลดลงตามความเข้มข้น โมดูลจึงให้แรงดันเอาต์พุตแบบแอนะล็อกที่สูงขึ้นเมื่อแอลกอฮอล์มากขึ้น เซนเซอร์ตรวจวัดแอลกอฮอล์ได้ในช่วงประมาณ 0.05–10 มิลลิกรัมต่อลิตร ใช้ขดลวดให้ความร้อนที่แรงดัน 5 โวลต์ ต้องอุ่นเซนเซอร์ (Preheat) ก่อนใช้งาน และค่าที่วัดได้ขึ้นกับอุณหภูมิและความชื้นของสภาพแวดล้อม (Hanwei Electronics, n.d.)

2.4 บอร์ดไมโครคอนโทรลเลอร์ ESP32

ESP32 เป็นไมโครคอนโทรลเลอร์ของบริษัท Espressif Systems ที่มีวงจร Wi-Fi และ Bluetooth ในตัว จึงเหมาะกับงาน IoT คุณสมบัติที่เกี่ยวข้องกับโครงงานแสดงดังตารางที่ 2.2

ตารางที่ 2.2 คุณสมบัติของ ESP32 ที่เกี่ยวข้องกับโครงงาน

คุณสมบัติ	รายละเอียด
หน่วยประมวลผล	Xtensa LX6 แบบ 32 บิต 2 แกน ความเร็วสูงสุด 240 MHz
หน่วยความจำ	SRAM 520 KB
การเชื่อมต่อไร้สาย	Wi-Fi 802.11 b/g/n และ Bluetooth
วงจรแปลงแอนะล็อกเป็นดิจิทัล	ADC แบบ SAR ความละเอียด 12 บิต (ค่า 0–4095) จำนวน 2 ชุด คือ ADC1 และ ADC2
การสื่อสารกับอุปกรณ์ต่อพ่วง	รองรับ I2C, SPI และ UART

ที่มา: Espressif Systems (n.d.-a)

ในการต่อเซนเซอร์ มีข้อจำกัดสำคัญ 2 ประการ คือ (1) ADC2 ถูกใช้งานโดยวงจร Wi-Fi ทำให้อ่านค่าไม่ได้ขณะเชื่อมต่อ Wi-Fi และ (2) ช่วงแรงดันที่อ่านได้ขึ้นกับค่าการลดทอนสัญญาณ (Attenuation) ที่ตั้งไว้ (Espressif Systems, n.d.-b) โครงงานนี้จึงเลือกขา GPIO 34 ซึ่งเป็นช่องของ ADC1 และตั้งค่าการลดทอนเป็น 11 dB เพื่อให้อ่านแรงดันได้กว้างที่สุด

2.5 จอแสดงผล OLED และการสื่อสารแบบ I2C

จอ OLED ขนาด 128×64 จุดที่ใช้ควบคุมด้วยชิป SSD1306 ซึ่งสื่อสารกับไมโครคอนโทรลเลอร์ผ่านบัส I2C โดยใช้สายสัญญาณเพียง 2 เส้น คือ SDA (ข้อมูล) และ SCL (สัญญาณนาฬิกา) และอ้างอิงอุปกรณ์ด้วยหมายเลขที่อยู่ (Address) ซึ่งโมดูลที่ใช้มีที่อยู่ 0x3C (Solomon Systech, 2008)

2.6 โปรโตคอล MQTT และแพลตฟอร์ม NETPIE

MQTT เป็นโปรโตคอลรับส่งข้อความแบบ Publish/Subscribe ผ่านตัวกลางที่เรียกว่า Broker ผู้ส่ง (Publisher) ส่งข้อความไปยังหัวข้อ (Topic) และผู้รับ (Subscriber) ที่สมัครรับหัวข้อนั้นจะได้รับข้อความทันที โดยไม่ต้องรู้จักกันโดยตรง MQTT กำหนดระดับการรับประกันการส่ง (Quality of Service: QoS) ไว้ 3 ระดับ คือ 0, 1 และ 2 และรองรับข้อความแบบ Retained ที่ Broker เก็บไว้ส่งให้ผู้รับรายใหม่ (Banks & Gupta, 2014)

NETPIE เป็นแพลตฟอร์ม IoT ที่ให้บริการ MQTT Broker โดยกำหนดให้หัวข้อสำหรับรับส่งข้อความขึ้นต้นด้วย @msg/ และมีบริการ Device Shadow สำหรับเก็บสถานะล่าสุดของอุปกรณ์ ซึ่งอุปกรณ์อัปเดตผ่านหัวข้อ @shadow/data/update นอกจากนี้ยังมี REST API ให้แอปพลิเคชันอ่าน Shadow และส่งข้อความไปยังอุปกรณ์ผ่าน HTTP ได้ (NETPIE, n.d.)

2.7 Firebase

Firebase เป็นแพลตฟอร์มบริการฝั่งเซิร์ฟเวอร์ของ Google โครงงานนี้ใช้บริการฐานข้อมูล 2 แบบ ได้แก่ Realtime Database ซึ่งเก็บข้อมูลเป็นโครงสร้าง JSON แบบต้นไม้ และ Cloud Firestore ซึ่งเก็บข้อมูลเป็นเอกสาร (Document) ภายในคอลเลกชัน (Collection) ทั้งสองบริการเข้าถึงได้ผ่าน REST API ด้วยคำสั่ง HTTP (Google, n.d.-a, n.d.-b)

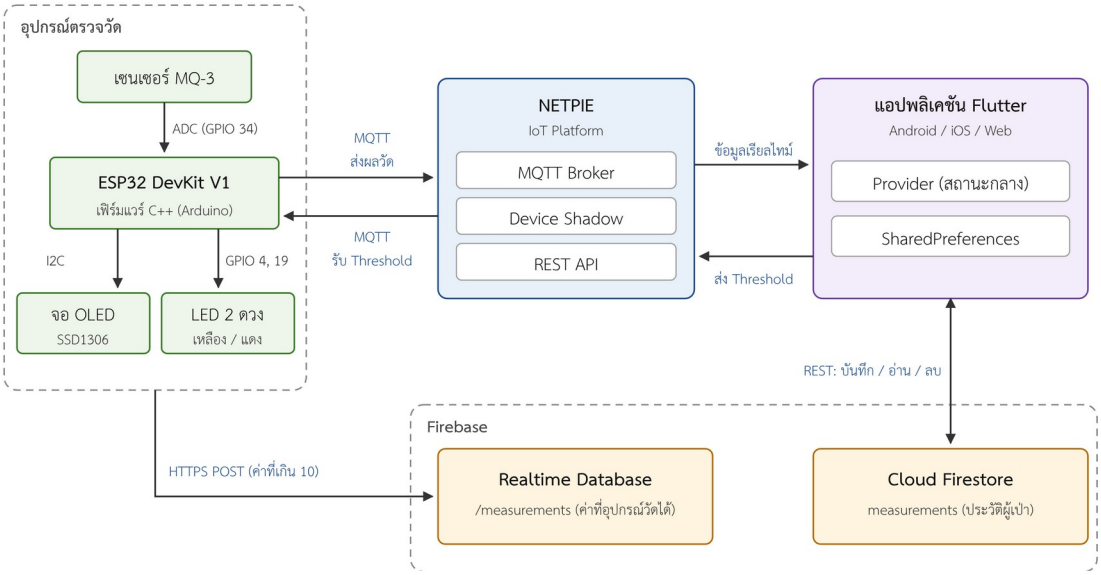
2.8 Flutter และการจัดการสถานะด้วย Provider

Flutter เป็นชุดพัฒนาซอฟต์แวร์ของ Google สำหรับสร้างแอปพลิเคชันหลายแพลตฟอร์ม เช่น Android, iOS และเว็บ จากซอร์สโค้ดชุดเดียวด้วยภาษา Dart (Google, n.d.-c) ส่วน Provider เป็นแพ็กเกจจัดการสถานะ (State Management) ที่ให้หน้าจอหลายหน้าใช้ข้อมูลชุดเดียวกัน และอัปเดตหน้าจอโดยอัตโนมัติเมื่อข้อมูลเปลี่ยน (Rousselet, n.d.)

บทที่ 3
การออกแบบและพัฒนาระบบ

3.1 สถาปัตยกรรมของระบบ

ระบบประกอบด้วย 4 ส่วนหลัก ได้แก่ อุปกรณ์ตรวจวัด แพลตฟอร์ม NETPIE แอปพลิเคชัน Flutter และฐานข้อมูล Firebase ซึ่งเชื่อมโยงกันดังภาพที่ 3.1 (Catboyz-45, 2026a, 2026b)



ภาพที่ 3.1 สถาปัตยกรรมของระบบตรวจวัดแอลกอฮอล์

ที่มา: คณะผู้จัดทำ

ลำดับการไหลของข้อมูลในระบบเป็นดังนี้

1. ESP32 อ่านค่าจากเซนเซอร์ MQ-3 ทุก 1 วินาที ประมวลผลเป็นค่าดัชนี 0–100 แสดงผลบนจอ OLED และควบคุมไฟ LED ตามระดับแจ้งเตือน
2. ESP32 ส่งสถานะล่าสุดไปยัง Device Shadow ทุกครั้งที่อ่านค่า และเมื่อค่าเกิน 10 จะส่งผลวัดผ่าน MQTT และบันทึกลง Firebase Realtime Database
3. แอปพลิเคชันรับค่าแบบเรียลไทม์ผ่าน MQTT และอ่าน Device Shadow ผ่าน REST API ทุก 3 วินาทีเป็นช่องทางสำรอง
4. ผู้ใช้กรอกข้อมูลผู้เข้ารับการตรวจในแอปพลิเคชัน แล้วบันทึกพร้อมผลวัดลง Cloud Firestore และหน่วยความจำในเครื่อง
5. ผู้ใช้ปรับค่า Threshold ในแอปพลิเคชัน ซึ่งส่งผ่าน REST API ของ NETPIE ไปยังอุปกรณ์ทางหัวข้อ MQTT ที่อุปกรณ์รอรับอยู่

3.2 อุปกรณ์ที่ใช้ในการพัฒนา

อุปกรณ์ฮาร์ดแวร์ที่ใช้ในการพัฒนาต้นแบบแสดงดังตารางที่ 3.1

ตารางที่ 3.1 อุปกรณ์ที่ใช้ในการพัฒนา

ลำดับ	อุปกรณ์	ภาพอุปกรณ์	จำนวน
1	**บอร์ด ESP32 DOIT DevKit V1** ไมโครคอนโทรลเลอร์พร้อม Wi-Fi ใช้ประมวลผลหลัก		1 บอร์ด
2	**เซนเซอร์ MQ-3** ตรวจวัดไอแอลกอฮอล์ในลมหายใจ		1 ตัว
3	**จอแสดงผล OLED 128×64 แบบ I2C** ชิปควบคุม SSD1306 ใช้แสดงค่าที่วัดได้		1 จอ
4	**หลอดไฟ LED** สีเหลืองและสีแดง ใช้แจ้งเตือนระดับ 1 และ 2		2 ดวง
5	**ตัวต้านทาน 220 โอห์ม** จำกัดกระแสของหลอดไฟ LED ดวงละ 1 ตัว		2 ตัว
6	**Breadboard** ใช้ต่อวงจรทดลอง		1 แผ่น
7	**สายจัมเปอร์** แบบผู้–เมีย ใช้เชื่อมต่ออุปกรณ์		2 ชุด

3.3 การต่อวงจร

การต่ออุปกรณ์เข้ากับขาของบอร์ด ESP32 เป็นไปตามค่าคงที่ที่กำหนดไว้ในเฟิร์มแวร์ ดังตารางที่ 3.2

ตารางที่ 3.2 การเชื่อมต่ออุปกรณ์กับขาของบอร์ด ESP32

อุปกรณ์	ขา ESP32	หน้าที่
MQ-3 (Analog Output)	GPIO 34	รับสัญญาณแอนะล็อกเข้า ADC1
LED สีเหลือง (ผ่านตัวต้านทาน 220 โอห์ม)	GPIO 4	แจ้งเตือนระดับ 1
LED สีแดง (ผ่านตัวต้านทาน 220 โอห์ม)	GPIO 19	แจ้งเตือนระดับ 2
OLED SDA	GPIO 21	สายข้อมูล I2C
OLED SCL	GPIO 22	สายสัญญาณนาฬิกา I2C

ที่มา: Catboyz-45 (2026b)

ข้อควรระวังในการต่อวงจร คือ ขา ADC ของ ESP32 รับแรงดันได้ไม่เกินแรงดันใช้งาน 3.3 โวลต์ หากจ่ายไฟให้โมดูล MQ-3 ที่ 5 โวลต์ แรงดันเอาต์พุตอาจสูงเกินช่วงดังกล่าว จึงควรใช้วงจรแบ่งแรงดัน (Voltage Divider) ก่อนต่อเข้าขา GPIO 34 (Espressif Systems, n.d.-a)

3.4 เครื่องมือและไลบรารีที่ใช้ในการพัฒนา

ฝั่งเฟิร์มแวร์พัฒนาบน PlatformIO ร่วมกับ Arduino Framework (PlatformIO Labs, n.d.) โดยใช้ไลบรารี PubSubClient สำหรับการเชื่อมต่อ MQTT (O'Leary, n.d.) และ ArduinoJson สำหรับสร้างและแยกข้อมูล JSON (Blanchon, n.d.) ส่วนฝั่งแอปพลิเคชันพัฒนาด้วย Flutter SDK เครื่องมือและเวอร์ชันทั้งหมดแสดงดังตารางที่ 3.3

ตารางที่ 3.3 เครื่องมือและไลบรารีที่ใช้ในการพัฒนา

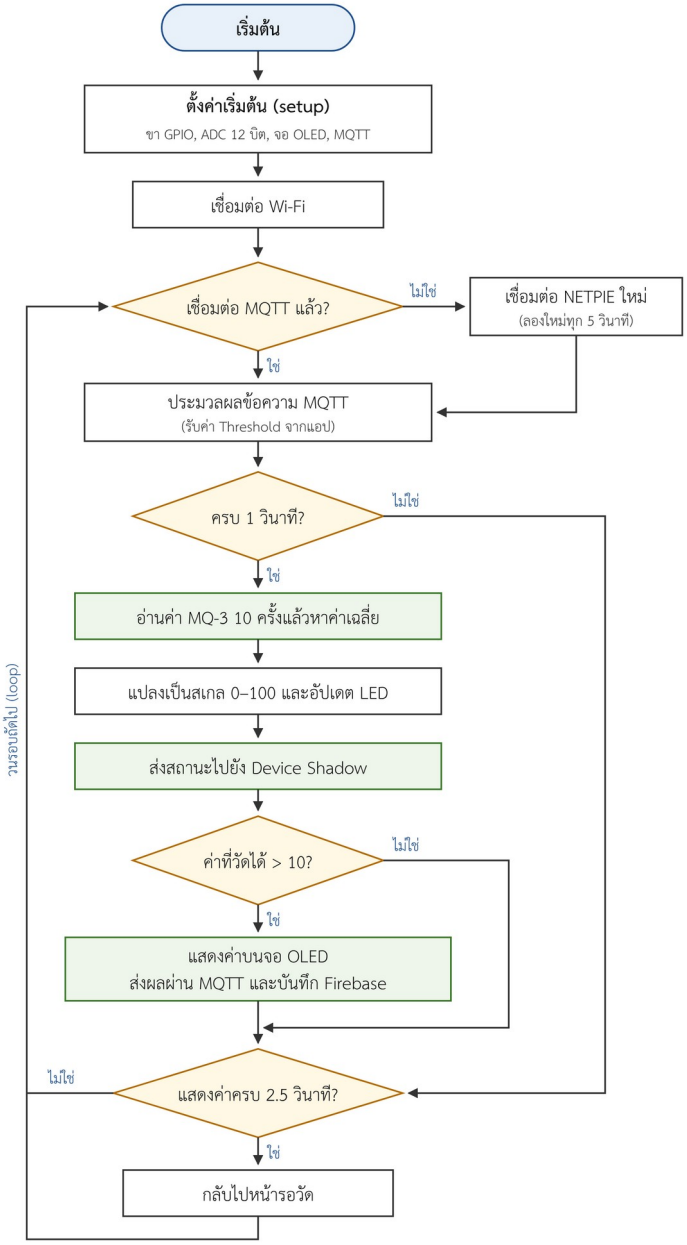
ส่วนงาน	เครื่องมือ / ไลบรารี	เวอร์ชัน	หน้าที่
ทั่วไป	Visual Studio Code	–	โปรแกรมแก้ไขโค้ดหลัก
เฟิร์มแวร์	PlatformIO, Arduino Framework	–	คอมไพล์และอัปโหลดโปรแกรมลงบอร์ด
เฟิร์มแวร์	PubSubClient	^2.8	เชื่อมต่อ MQTT
เฟิร์มแวร์	ArduinoJson	^6.21.3	สร้างและแยกข้อมูล JSON
เฟิร์มแวร์	Adafruit SSD1306, Adafruit GFX	^2.5.10, ^1.11.10	ควบคุมจอ OLED
แอปพลิเคชัน	Flutter SDK (Dart)	^3.7.2	พัฒนาแอปพลิเคชันหลายแพลตฟอร์ม
แอปพลิเคชัน	provider	^6.1.5	จัดการสถานะกลาง
แอปพลิเคชัน	mqtt_client	^10.8.0	รับข้อมูลผ่าน MQTT
แอปพลิเคชัน	http	^1.5.0	เรียก REST API ของ NETPIE และ Firestore
แอปพลิเคชัน	shared_preferences	^2.5.3	เก็บประวัติไว้ในเครื่อง
บริการคลาวด์	NETPIE, Firebase	–	รับส่งข้อมูลและจัดเก็บข้อมูล

ที่มา: ไฟล์ platformio.ini และ pubspec.yaml ของโครงงาน (Catboyz-45, 2026a, 2026b)

3.5 การพัฒนาเฟิร์มแวร์

3.5.1 ลำดับการทำงานของเฟิร์มแวร์

เฟิร์มแวร์ทำงานแบบไม่หยุดรอ (Non-blocking) โดยใช้ฟังก์ชัน millis() จับเวลาแทนการหน่วงเวลา ทำให้รับคำสั่งจาก MQTT ได้ตลอดเวลาระหว่างรอรอบการอ่านค่า ลำดับการทำงานแสดงดังภาพที่ 3.2



ภาพที่ 3.2 ผังงานการทำงานของเฟิร์มแวร์บน ESP32

3.5.2 การอ่านค่าและแปลงค่าจากเซนเซอร์

ในแต่ละรอบ เฟิร์มแวร์อ่านค่าจากขา GPIO 34 จำนวน 10 ครั้ง ห่างกันครั้งละ 5 มิลลิวินาที แล้วหาค่าเฉลี่ยเพื่อลดสัญญาณรบกวน ได้ค่าดิบ (Raw) ในช่วง 0–4095 ตามความละเอียด 12 บิตของ ADC จากนั้นแปลงเป็นค่าดัชนีแอลกอฮอล์ในสเกล 0–100 แบบเชิงเส้น จำกัดค่าไม่ให้เกินช่วง และปัดเป็นทศนิยม 1 ตำแหน่ง ตามสูตร

ค่าดัชนีแอลกอฮอล์ = (ค่าดิบ ÷ 4095) × 100

ตัวอย่างโค้ดที่ 3.1 ฟังก์ชันอ่านค่าและแปลงค่าจากเซนเซอร์

```
// อ่านค่าเซ็นเซอร์หลายครั้งแล้วเฉลี่ย เพื่อลดสัญญาณรบกวน
int readAlcoholRaw() {
  long total = 0;
  const int sampleCount = 10;

  for (int sample = 0; sample < sampleCount; sample++) {
    total += analogRead(MQ3_PIN);
    delay(5);
  }

  return total / sampleCount;
}

// แปลงค่าดิบเป็นเปอร์เซ็นต์ ตามสเกล 0-100
float convertRawToBac(int rawValue) {
  return clampBac((rawValue / 4095.0f) * BAC_MAX_VALUE);
}
```

3.5.3 การแบ่งระดับการแจ้งเตือน

ระบบเปรียบเทียบค่าที่วัดได้กับ Threshold 2 ค่า ค่าเริ่มต้นคือ 20 และ 50 เพื่อแบ่งระดับการแจ้งเตือนเป็น 3 ระดับ ดังตารางที่ 3.4 เมื่อได้รับ Threshold ใหม่ เฟิร์มแวร์จะจำกัดค่าให้อยู่ในช่วง 0–100 และสลับค่าให้ Threshold 1 ไม่มากกว่า Threshold 2 เสมอ

ตารางที่ 3.4 ระดับการแจ้งเตือน

ระดับ	เงื่อนไข	LED เหลือง	LED แดง	สถานะในแอป
0	ค่า < Threshold 1	ดับ	ดับ	SAFE
1	Threshold 1 ≤ ค่า < Threshold 2	ติด	ดับ	WARNING
2	ค่า ≥ Threshold 2	ติด	ติด	DRUNK

ตัวอย่างโค้ดที่ 3.2 ฟังก์ชันอัปเดตระดับแจ้งเตือนและควบคุม LED

```
// อัปเดตระดับแจ้งเตือน และควบคุม LED ตามค่า BAC
void updateAlertOutputs(float bacValue) {
  if (bacValue >= threshold2) {
    alertLevel = 2;
  } else if (bacValue >= threshold1) {
    alertLevel = 1;
  } else {
    alertLevel = 0;
  }

  digitalWrite(LED1_PIN, alertLevel >= 1 ? HIGH : LOW);
  digitalWrite(LED2_PIN, alertLevel >= 2 ? HIGH : LOW);
}
```

3.5.4 การแสดงผลบนจอ OLED

จอ OLED แสดงข้อมูล 4 บรรทัด ได้แก่ ชื่อระบบ ค่าที่วัดได้ด้วยตัวอักษรขนาดใหญ่ ค่า Threshold ทั้งสอง และสถานะการทำงาน เมื่อวัดได้ค่าเกิน 10 จอจะค้างค่าที่วัดได้ไว้ 2.5 วินาที พร้อมข้อความ "Sent to cloud" จากนั้นกลับไปหน้ารอวัดที่แสดงข้อความ "Waiting for sample"

3.5.5 ค่าคงที่สำคัญในเฟิร์มแวร์

ตารางที่ 3.5 ค่าคงที่สำคัญในเฟิร์มแวร์

ค่าคงที่	ค่า	ความหมาย
SENSOR_READ_INTERVAL_MS	1,000 ms	ช่วงเวลาระหว่างการอ่านเซนเซอร์แต่ละรอบ
DISPLAY_HOLD_MS	2,500 ms	ระยะเวลาค้างค่าที่วัดได้บนจอ
MIN_REPORT_BAC	10	ค่าขั้นต่ำที่ถือว่ามีการเป่าและส่งข้อมูลขึ้นคลาวด์
BAC_MAX_VALUE	100	ค่าสูงสุดของสเกล
threshold1, threshold2	20, 50	ค่าเริ่มต้นของ Threshold
MQTT Buffer Size	512 bytes	ขนาดบัฟเฟอร์ข้อความ MQTT

3.6 การสื่อสารข้อมูลผ่าน MQTT

อุปกรณ์เชื่อมต่อ Broker ของ NETPIE (mqtt.netpie.io) และใช้หัวข้อต่าง ๆ ดังตารางที่ 3.6 หากการเชื่อมต่อหลุด เฟิร์มแวร์จะพยายามเชื่อมต่อใหม่ทุก 5 วินาที

ตารางที่ 3.6 หัวข้อ MQTT ที่ใช้ในระบบ

ทิศทาง	หัวข้อ (Topic)	หน้าที่
รับ	@msg/home/device_control	รับค่า Threshold จากแอปพลิเคชัน
ส่ง	@msg/home/device_status	ส่งผลวัดและสถานะของอุปกรณ์
ส่ง	@msg/home/bac	ส่งผลวัดสำหรับการติดตามค่า
ส่ง	@shadow/data/update	อัปเดต Device Shadow (ส่งแบบ Retained)

ที่มา: Catboyz-45 (2026b)

ตัวอย่างโค้ดที่ 3.3 ข้อมูลผลวัดที่อุปกรณ์ส่ง (JSON)

```
{
  "bac": 32.5,
  "raw": 1331,
  "threshold1": 20,
  "threshold2": 50,
  "alertLevel": 1,
  "millis": 120000
}
```

ข้อมูลข้างต้นประกอบด้วยค่าดัชนีแอลกอฮอล์ ค่าดิบจาก ADC ค่า Threshold ทั้งสองระดับการแจ้งเตือน และเวลาที่ผ่านไปนับจากบอร์ดเริ่มทำงาน ส่วนคำสั่งปรับ Threshold ใช้ JSON ในรูปแบบ {"threshold1": 20, "threshold2": 50} โดยเฟิร์มแวร์รองรับชื่อฟิลด์ level1 และ level2 รวมถึงข้อมูลที่ซ้อนอยู่ในฟิลด์ data ด้วย

3.7 การออกแบบการจัดเก็บข้อมูล

3.7.1 Firebase Realtime Database

อุปกรณ์บันทึกผลวัดที่เกินค่าขั้นต่ำด้วยคำสั่ง HTTP POST ไปยังโหนด /measurements ซึ่ง Firebase สร้างคีย์ไม่ซ้ำให้แต่ละรายการโดยอัตโนมัติ (Google, n.d.-b) ฟิลด์ที่บันทึกเหมือนกับข้อมูลในตัวอย่างโค้ดที่ 3.3

3.7.2 Cloud Firestore

แอปพลิเคชันบันทึกประวัติผู้เข้ารับการตรวจในคอลเลกชัน measurements โดยใช้เวลาที่บันทึกในหน่วยมิลลิวินาทีเป็นรหัสเอกสาร โครงสร้างข้อมูลแสดงดังตารางที่ 3.7

ตารางที่ 3.7 โครงสร้างเอกสารในคอลเลกชัน measurements

ฟิลด์	ชนิดข้อมูล	ความหมาย
id	string	รหัสรายการ (เวลาที่บันทึกในหน่วยมิลลิวินาที)
name	string	ชื่อ-นามสกุลผู้เข้ารับการตรวจ
idCard	string	เลขบัตรประชาชน 13 หลัก
age	integer	อายุ (ปี)
licensePlate	string	ทะเบียนรถ
province	string	จังหวัดของทะเบียนรถ
alcoholValue	double	ค่าดัชนีแอลกอฮอล์ที่วัดได้
status	string	สถานะ SAFE, WARNING หรือ DRUNK
timestamp	string	วันและเวลาที่บันทึกในรูปแบบ ISO 8601

ที่มา: Catboyz-45 (2026a)

3.7.3 การจัดเก็บข้อมูลในเครื่อง

แอปพลิเคชันออกแบบให้ทำงานได้แม้ไม่มีอินเทอร์เน็ต (Offline-first) เมื่อเปิดหน้าประวัติ แอปจะดึงข้อมูลจาก Firestore ก่อน หากไม่สำเร็จจะใช้ข้อมูลที่เก็บไว้ในเครื่องด้วย SharedPreferences และเมื่อบันทึกหรือลบรายการ แอปจะพยายามปรับปรุงข้อมูลบน Firestore ก่อน แล้วปรับปรุงข้อมูลในเครื่องเสมอ

3.8 การพัฒนาแอปพลิเคชัน

3.8.1 โครงสร้างซอร์สโค้ด

ตารางที่ 3.8 โครงสร้างซอร์สโค้ดของแอปพลิเคชัน

ไฟล์	หน้าที่
main.dart	จุดเริ่มต้นของแอปพลิเคชัน สร้าง Provider กลางและกำหนดธีม
app_state.dart	เก็บสถานะกลาง โมเดลข้อมูล และจัดการประวัติกับ Firestore และ SharedPreferences
home_page.dart	จัดการแถบเมนู เชื่อมต่อ MQTT และดึงสถานะผ่าน HTTP ทุก 3 วินาที
home_service.dart	เรียก REST API ของ NETPIE เพื่ออ่าน Shadow และส่ง Threshold
mqtt_service.dart	เชื่อมต่อ MQTT และแปลงข้อมูลที่ได้รับ
mqtt_client_factory*.dart	เลือกการเชื่อมต่อ MQTT ตามแพลตฟอร์ม (TCP สำหรับแอป และ WebSocket สำหรับเว็บ)
measurement_form_page.dart	หน้าวัดค่าและแบบฟอร์มบันทึกข้อมูลผู้เข้ารับการตรวจ
measurement_history_page.dart	หน้าประวัติการตรวจวัดและการลบรายการ
settings_page.dart	หน้าตั้งค่า Threshold สถานะการเชื่อมต่อ และมาตรวัดค่า

ที่มา: Catboyz-45 (2026a)

3.8.2 การรับข้อมูลจากอุปกรณ์

แอปพลิเคชันรับข้อมูลจากอุปกรณ์ 2 ช่องทางเพื่อความต่อเนื่อง ช่องทางหลักคือ MQTT ซึ่งสมัครรับหัวข้อ @shadow/data/update และ @msg/home/device_status แบบ QoS 0 โดยแอปบนมือถือเชื่อมต่อผ่าน TCP พอร์ต 1883 และแอปบนเว็บเชื่อมต่อผ่าน WebSocket แบบเข้ารหัสที่พอร์ต 443 ช่องทางสำรองคือการอ่าน Device Shadow ผ่าน REST API ของ NETPIE ทุก 3 วินาที ข้อมูลจากทั้งสองช่องทางจะถูกส่งเข้าสถานะกลางชุดเดียวกัน ทำให้ทุกหน้าจออัปเดตตรงกัน นอกจากนี้ หน้าวัดค่าจะแสดงค่าที่ต่ำกว่า 10 เป็น 0 เพื่อตัดสัญญาณรบกวนเมื่อไม่มีการเป่า

3.8.3 หน้าจอของแอปพลิเคชัน

1. หน้าวัดค่า แสดงค่าที่วัดได้ล่าสุด สถานะ และค่า Threshold พร้อมแบบฟอร์มบันทึกข้อมูลผู้เข้ารับการตรวจ ซึ่งตรวจสอบข้อมูลก่อนบันทึกตามตารางที่ 3.9
2. หน้าประวัติ แสดงรายการตรวจวัดทั้งหมดเรียงจากล่าสุด พร้อมสีและไอคอนตามสถานะ ปุ่มรีเฟรช และการลบรายการที่มีกล่องยืนยันก่อนลบ
3. หน้าตั้งค่า แสดงสถานะการเชื่อมต่อ MQTT แถบเลื่อนปรับ Threshold ทั้งสองในช่วง 1–100 ปุ่มส่งค่ามาตรฐานไปยังอุปกรณ์ และมาตรวัดแบบหน้าปัดที่แบ่งสีตามระดับ

ตารางที่ 3.9 เงื่อนไขการตรวจสอบข้อมูลในแบบฟอร์ม

ข้อมูล	เงื่อนไข
ชื่อ-นามสกุล	ต้องไม่เป็นค่าว่าง
เลขบัตรประชาชน	ต้องไม่เป็นค่าว่าง และต้องมี 13 หลัก
อายุ	ต้องเป็นตัวเลขจำนวนเต็มระหว่าง 1–120 ปี
ทะเบียนรถ	ต้องไม่เป็นค่าว่าง
จังหวัด	ต้องเลือกจากรายการจังหวัดในภาคกลาง 19 จังหวัด

ที่มา: Catboyz-45 (2026a)

3.8.4 การส่งค่า Threshold ไปยังอุปกรณ์

เมื่อผู้ใช้กดส่งค่ามาตรฐาน แอปจะจำกัดค่าให้อยู่ในช่วง 0–100 จัดลำดับให้ค่าน้อยเป็น Threshold 1 แล้วส่งคำสั่ง HTTP PUT ไปยัง REST API ของ NETPIE ที่หัวข้อ home/device_control ซึ่ง NETPIE จะส่งต่อเป็นข้อความ MQTT ไปยังอุปกรณ์ทางหัวข้อ @msg/home/device_control จากนั้นอุปกรณ์จะอัปเดต Device Shadow ทำให้แอปได้รับค่าที่ใช้งานจริงกลับมา

3.8.5 การจัดการข้อมูลสำหรับเชื่อมต่อ

ข้อมูลสำหรับเชื่อมต่อ เช่น รหัส Wi-Fi รหัสอุปกรณ์ NETPIE และที่อยู่ Firebase ถูกแยกออกจากซอร์สโค้ด ฝั่งเฟิร์มแวร์เก็บไว้ในไฟล์ secrets.h และฝั่งแอปพลิเคชันส่งผ่านไฟล์ config.json ด้วยคำสั่ง --dart-define-from-file โดยไฟล์ทั้งสองถูกยกเว้นไม่ให้อัปโหลดขึ้นที่เก็บโค้ด

3.9 ขั้นตอนการพัฒนา

1. ต่อวงจรบน Breadboard โดยต่อเซนเซอร์ MQ-3 เข้ากับขา Analog ของ ESP32 ต่อจอ OLED ผ่าน I2C และต่อหลอดไฟ LED ผ่านตัวต้านทาน

2. เขียนเฟิร์มแวร์ให้อ่านค่าจากเซนเซอร์ แสดงผลบนจอ OLED และควบคุมไฟ LED ตามระดับแจ้งเตือน
3. ตั้งค่าให้ ESP32 เชื่อมต่อ Wi-Fi และ NETPIE ผ่าน MQTT และส่งข้อมูลไปยัง Firebase
4. ออกแบบโครงสร้างการจัดเก็บข้อมูล ให้รองรับข้อมูลผู้เข้ารับการตรวจและค่าระดับแอลกอฮอล์ที่วัดได้
5. พัฒนาแอปพลิเคชันให้รับข้อมูลการเป่ามาแสดงผล บันทึกประวัติ และปรับค่า Threshold
6. ทดสอบความถูกต้องของการแสดงผลบนจอ OLED สถานะของไฟ LED และการรับส่งข้อมูลกับแอปพลิเคชัน
7. ออกแบบตัวเครื่องสำหรับบรรจุอุปกรณ์

บทที่ 4
ผลการดำเนินงาน

4.1 ต้นแบบอุปกรณ์

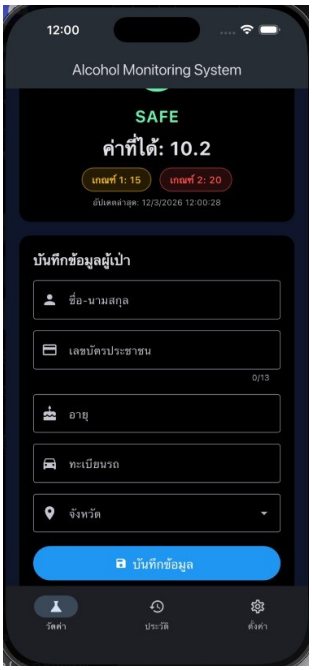
ต้นแบบอุปกรณ์ตรวจวัดแอลกอฮอล์ประกอบอยู่ในตัวเครื่อง โดยมีจอ OLED แสดงค่าที่วัดได้และไฟ LED แจ้งเตือนอยู่ด้านหน้า ดังภาพที่ 4.1



ภาพที่ 4.1 ต้นแบบอุปกรณ์ตรวจวัดแอลกอฮอล์

4.2 แอปพลิเคชัน

ภาพที่ 4.2 แสดงหน้าวัดค่าซึ่งแสดงสถานะ SAFE ค่าที่วัดได้ 10.2 และค่า Threshold พร้อมแบบฟอร์มบันทึกข้อมูล และหน้าประวัติซึ่งแสดงรายการตรวจวัดที่มีสถานะต่างกัน ส่วนภาพที่ 4.3 แสดงหน้าตั้งค่าที่ปรับ Threshold เป็น 15 และ 20 และมาตรวัดแบบหน้าปัด

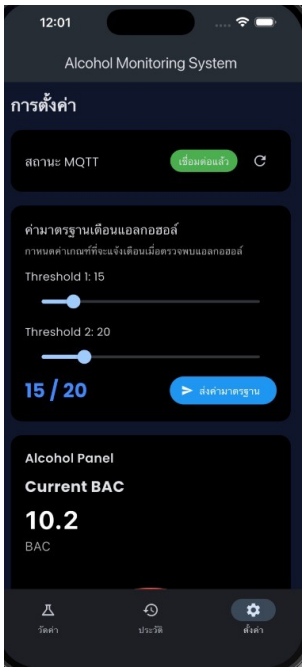


หน้าวัดค่า

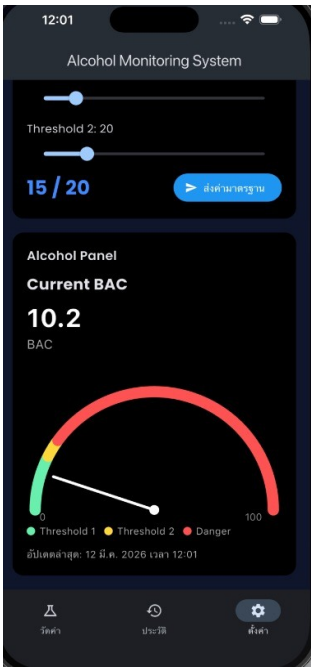


หน้าประวัติ

ภาพที่ 4.2 หน้าวัดค่าและหน้าประวัติของแอปพลิเคชัน



หน้าตั้งค่า

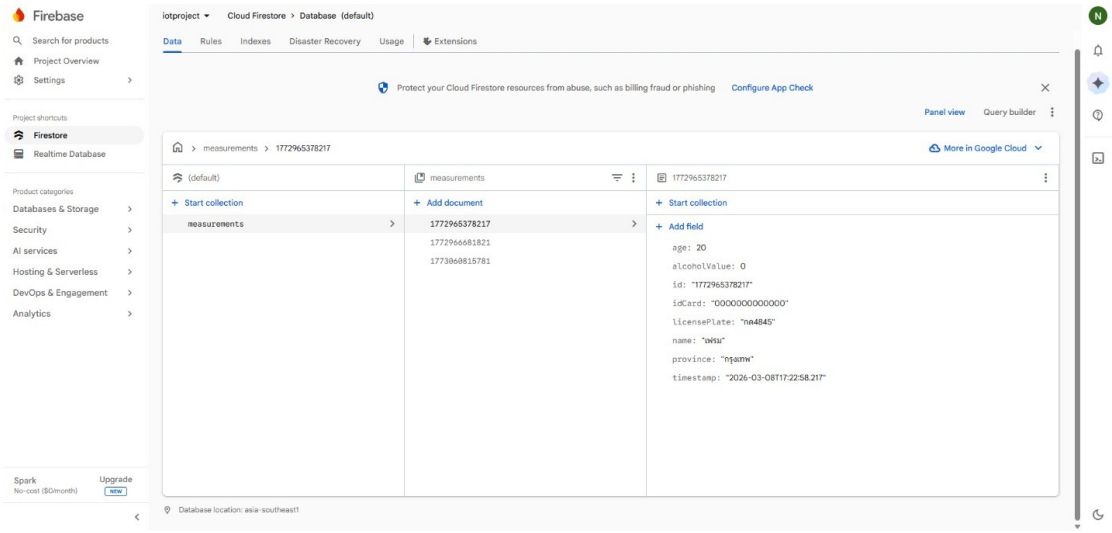


มาตรวัดค่าในหน้าตั้งค่า

ภาพที่ 4.3 หน้าตั้งค่าของแอปพลิเคชัน

4.3 ข้อมูลบน Cloud Firestore

ภาพที่ 4.4 แสดงเอกสารในคอลเลกชัน measurements บน Cloud Firestore ซึ่งมีฟิลด์ตรงตามโครงสร้างในตารางที่ 3.7



ภาพที่ 4.4 ข้อมูลการตรวจวัดที่บันทึกบน Cloud Firestore

4.4 ผลการทดสอบ

คณะผู้จัดทำทดสอบการทำงานของระบบตามฟังก์ชันหลัก ผลการทดสอบแสดงดังตารางที่ 4.1

ตารางที่ 4.1 ผลการทดสอบการทำงานของระบบ

ลำดับ	รายการทดสอบ	ผลที่คาดหวัง	ผลการทดสอบ
1	อ่านค่าเซนเซอร์และแสดงผลบนจอ OLED	จอแสดงค่าที่วัดได้และสถานะ	ผ่าน (ภาพที่ 4.1)
2	รับค่าจากอุปกรณ์แบบเรียลไทม์	แอปแสดงค่าและสถานะล่าสุด	ผ่าน (ภาพที่ 4.2)
3	บันทึกข้อมูลผู้เข้ารับการตรวจ	รายการใหม่ปรากฏในหน้าประวัติ	ผ่าน (ภาพที่ 4.2)
4	แบ่งสถานะตาม Threshold	แสดง SAFE, WARNING และ DRUNK ตามค่าที่วัดได้	ผ่าน (ภาพที่ 4.2)
5	ปรับและส่งค่า Threshold	ค่าใหม่แสดงในหน้าตั้งค่าและมาตรวัด	ผ่าน (ภาพที่ 4.3)
6	จัดเก็บข้อมูลบน Cloud Firestore	มีเอกสารในคอลเลกชัน measurements	ผ่าน (ภาพที่ 4.4)

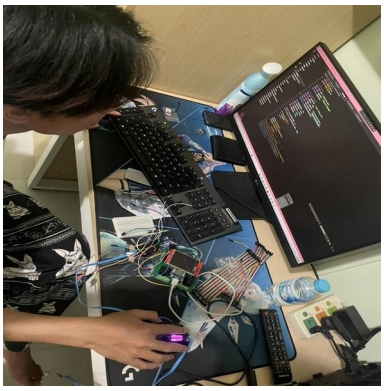
นอกจากนี้ ได้ตรวจสอบคุณภาพของซอร์สโค้ดด้วยเครื่องมือของแต่ละแพลตฟอร์ม ผลแสดงดังตารางที่ 4.2

ตารางที่ 4.2 ผลการตรวจสอบการคอมไพล์และคุณภาพของซอร์สโค้ด

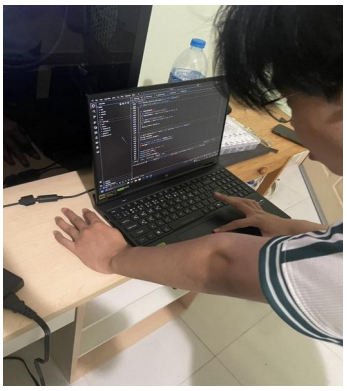
ส่วนงาน	คำสั่ง	ผลการตรวจสอบ
เฟิร์มแวร์	pio run	ผ่าน ใช้ RAM ร้อยละ 14.5 (47,648 จาก 327,680 ไบต์) และ Flash ร้อยละ 73.6 (964,757 จาก 1,310,720 ไบต์)
แอปพลิเคชัน	flutter analyze	ผ่าน ไม่พบปัญหา
แอปพลิเคชัน	flutter test	ผ่าน 1 การทดสอบ
แอปพลิเคชัน	flutter build apk --release	ผ่าน

ที่มา: Catboyz-45 (2026a, 2026b)

4.5 ภาพการดำเนินงาน



การเขียนโปรแกรมและต่อวงจร



การพัฒนาซอร์สโค้ด



การประกอบอุปกรณ์



การทดสอบอุปกรณ์

ภาพที่ 4.5 ภาพการดำเนินงานของคณะผู้จัดทำ

บทที่ 5
สรุปผล อภิปรายผล และข้อเสนอแนะ

5.1 สรุปผล

โครงงานระบบตรวจวัดแอลกอฮอล์ด้วยเทคโนโลยี IoT ดำเนินการได้ตามวัตถุประสงค์ ดังนี้

1. ได้ระบบที่ใช้ ESP32 ร่วมกับเซนเซอร์ MQ-3 อ่านค่าแอลกอฮอล์ในลมหายใจทุก 1 วินาที เฉลี่ยค่าจาก 10 ตัวอย่าง และแบ่งการแจ้งเตือนเป็น 3 ระดับ
2. แสดงผลแบบเรียลไทม์ได้ทั้งบนจอ OLED ไฟ LED และแอปพลิเคชัน โดยแอปรับข้อมูลผ่าน MQTT และมีการอ่าน Device Shadow ทุก 3 วินาทีเป็นช่องทางสำรอง
3. จัดเก็บข้อมูลบน Firebase ได้ 2 ส่วน คือ ผลวัดจากอุปกรณ์บน Realtime Database และประวัติผู้เข้ารับการตรวจบน Cloud Firestore พร้อมสำเนาในเครื่อง
4. ได้ประยุกต์ใช้ ESP32 ร่วมกับแพลตฟอร์ม NETPIE ทั้ง MQTT, Device Shadow และ REST API รวมถึงพัฒนาแอปพลิเคชันหลายแพลตฟอร์มด้วย Flutter
5. ได้เครื่องมือคัดกรองเบื้องต้นที่ปรับเกณฑ์ได้ตามกลุ่มผู้ขับขี่ และเก็บประวัติเพื่อต่อยอดเป็นอุปกรณ์พกพาหรือใช้ที่จุดคัดกรอง

5.2 อภิปรายผล

การออกแบบให้ปรับ Threshold ได้จากแอปพลิเคชันช่วยให้ระบบรองรับเกณฑ์ตามกฎหมายที่ต่างกันระหว่างกลุ่มผู้ขับขี่ในตารางที่ 2.1 อย่างไรก็ตาม ค่าที่ระบบแสดงเป็นการแปลงค่าดิบจาก ADC แบบเชิงเส้น ขณะที่ความต้านทานของเซนเซอร์ MQ-3 เปลี่ยนแปลงไม่เป็นเชิงเส้นตามความเข้มข้นของแอลกอฮอล์ (Hanwei Electronics, n.d.) ค่าดัชนีจึงใช้เปรียบเทียบระดับมากหรือน้อยได้ แต่ยังไม่สามารถเทียบเป็นมิลลิกรัมเปอร์เซ็นต์ได้โดยตรง

ในด้านการสื่อสาร การใช้ MQTT ร่วมกับการอ่าน Device Shadow เป็นระยะช่วยให้แอปแสดงค่าล่าสุดได้แม้การเชื่อมต่อ MQTT ขาดช่วง และการออกแบบแบบ Offline-first ช่วยให้บันทึกข้อมูลได้แม้ไม่มีอินเทอร์เน็ต

5.3 ปัญหาและข้อจำกัด

1. ระบบยังไม่ได้สอบเทียบกับเครื่องวัดแอลกอฮอล์มาตรฐาน จึงใช้ได้เพื่อการคัดกรองเบื้องต้นเท่านั้น ไม่สามารถใช้เป็นหลักฐานทางกฎหมาย
2. ค่าที่วัดได้จากเซนเซอร์ MQ-3 ขึ้นกับอุณหภูมิ ความชื้น และระยะเวลาการอุ่นเซนเซอร์
3. การเชื่อมต่อ MQTT ของอุปกรณ์ใช้พอร์ต 1883 ซึ่งไม่เข้ารหัส และการเชื่อมต่อ Firebase ยังไม่ตรวจสอบใบรับรอง TLS

4. แอปพลิเคชันเก็บข้อมูลสำหรับเชื่อมต่อ NETPIE และ Firebase ไว้ในตัวแอป จึงอาจถูกดึงออกไปใช้ได้
5. ระบบจัดเก็บชื่อและเลขบัตรประชาชน ซึ่งเป็นข้อมูลส่วนบุคคลตามพระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562 การใช้งานจริงจึงต้องมีการยืนยันตัวตน การกำหนดสิทธิ์การเข้าถึง และการขอความยินยอมจากเจ้าของข้อมูล (พระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล, 2562)
6. หน้าตั้งค่าของแอปพลิเคชันมีคำสั่งเปิด–ปิด Buzzer และไฟ LED แต่เฟิร์มแวร์ปัจจุบันรองรับเฉพาะคำสั่งปรับ Threshold คำสั่งดังกล่าวจึงยังไม่มีผลกับอุปกรณ์

5.4 ข้อเสนอแนะ

1. สอบเทียบเซนเซอร์กับเครื่องวัดมาตรฐาน และสร้างสูตรแปลงค่าจากข้อมูลการทดลองจริงเพื่อแสดงผลเป็นหน่วยที่ตรวจสอบได้
2. เปลี่ยนการเชื่อมต่อ MQTT เป็นแบบ TLS และตรวจสอบใบรับรองของเซิร์ฟเวอร์ทั้ง NETPIE และ Firebase
3. เพิ่มเซิร์ฟเวอร์กลาง (Backend) ที่ตรวจสอบสิทธิ์ผู้ใช้ เพื่อไม่ต้องเก็บข้อมูลสำหรับเชื่อมต่อไว้ในแอปพลิเคชัน และกำหนด Firestore Security Rules
4. เพิ่มระบบสมาชิก การกำหนดสิทธิ์ตามบทบาท และการปกปิดเลขบัตรประชาชนบางส่วนเมื่อแสดงผล
5. เพิ่มการรองรับคำสั่ง Buzzer และไฟ LED ในเฟิร์มแวร์ พร้อมการตอบกลับยืนยันเมื่อได้รับคำสั่ง
6. เพิ่มวงจรแบ่งแรงดันระหว่างเซนเซอร์กับ ESP32 การจัดเก็บข้อมูลชั่วคราวเมื่ออินเทอร์เน็ตขาด และการอัปเดตเฟิร์มแวร์ผ่านเครือข่าย (OTA)

บรรณานุกรม

กฎกระทรวง ฉบับที่ 21 (พ.ศ. 2560) ออกตามความในพระราชบัญญัติจราจรทางบก พ.ศ. 2522. (2560). ราชกิจจานุเบกษา.

พระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562. (2562, 27 พฤษภาคม). ราชกิจจานุเบกษา. เล่ม 136 ตอนที่ 69 ก.

มติชนออนไลน์. (2560, 31 พฤษภาคม). ประกาศใช้ กม.เมาแล้วขับ เด็กต่ำกว่า 20 ปี–ใบขับขี่ชั่วคราว แอลกอฮอล์เกิน 20 มก. ถูกจับ. สืบค้นจาก https://www.matichon.co.th/local/news_571560

Banks, A., & Gupta, R. (Eds.). (2014). MQTT version 3.1.1 (OASIS Standard). OASIS. Retrieved from http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

Blanchon, B. (n.d.). ArduinoJson. Retrieved from https://arduinojson.org

Catboyz-45. (2026a). alcohol-monitor-app [Source code]. GitHub. Retrieved from https://github.com/Catboyz-45/alcohol-monitor-app

Catboyz-45. (2026b). alcohol-monitor-firmware [Source code]. GitHub. Retrieved from https://github.com/Catboyz-45/alcohol-monitor-firmware

Espressif Systems. (n.d.-a). ESP32 series datasheet. Retrieved from https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

Espressif Systems. (n.d.-b). Analog to digital converter (ADC). ESP-IDF programming guide. Retrieved from https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/adc_oneshot.html

Google. (n.d.-a). Use the Cloud Firestore REST API. Firebase documentation. Retrieved from https://firebase.google.com/docs/firestore/use-rest-api

Google. (n.d.-b). Firebase Realtime Database REST API. Firebase documentation. Retrieved from https://firebase.google.com/docs/database/rest/start

Google. (n.d.-c). Flutter documentation. Retrieved from https://docs.flutter.dev

Hanwei Electronics. (n.d.). Technical data MQ-3 gas sensor. Retrieved from https://www.sparkfun.com/datasheets/Sensors/MQ-3.pdf

International Telecommunication Union. (2012). Overview of the Internet of things (ITU-T Recommendation Y.2060). Retrieved from https://www.itu.int/rec/T-REC-Y.2060-201206-I

NETPIE. (n.d.). NETPIE documentation. Retrieved from https://docs.netpie.io

O'Leary, N. (n.d.). Arduino Client for MQTT (PubSubClient). Retrieved from https://pubsubclient.knolleary.net

PlatformIO Labs. (n.d.). PlatformIO documentation. Retrieved from https://docs.platformio.org

Rousselet, R. (n.d.). provider [Dart package]. pub.dev. Retrieved from https://pub.dev/packages/provider

Solomon Systech. (2008). SSD1306: 128 x 64 dot matrix OLED/PLED segment/common driver with controller (Rev. 1.1). Retrieved from https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf

ภาคผนวก ก
ที่เก็บซอร์สโค้ดและวิธีติดตั้ง

ซอร์สโค้ดของโครงงานเผยแพร่บน GitHub แยกเป็น 2 ที่เก็บ ได้แก่

1. เฟิร์มแวร์ ESP32: https://github.com/Catboyz-45/alcohol-monitor-firmware
2. แอปพลิเคชัน Flutter: https://github.com/Catboyz-45/alcohol-monitor-app

ก.1 การติดตั้งเฟิร์มแวร์

```
git clone https://github.com/Catboyz-45/alcohol-monitor-firmware.git
cd alcohol-monitor-firmware
cp include/secrets.example.h include/secrets.h
# แก้ไข secrets.h ให้เป็นค่า Wi-Fi, NETPIE และ Firebase ของตนเอง
pio run --target upload
pio device monitor --baud 115200
```

ก.2 การติดตั้งแอปพลิเคชัน

```
git clone https://github.com/Catboyz-45/alcohol-monitor-app.git
cd alcohol-monitor-app
flutter pub get
cp config.example.json config.json
# แก้ไข config.json ให้เป็นค่า NETPIE และ Firebase ของตนเอง
flutter run --dart-define-from-file=config.json
```

ภาคผนวก ข

ซอร์สโค้ดเฟิร์มแวร์ (src/main.cpp)

```
#include <Arduino.h>
#include <ArduinoJson.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <HTTPClient.h>
#include <PubSubClient.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <Wire.h>

#include "secrets.h"

const char *mqttServer = "mqtt.netpie.io";
const int mqttPort = 1883;

const char *topic_sub = "@msg/home/device_control";
const char *topic_pub = "@msg/home/device_status";
const char *topic_pub_bac = "@msg/home/bac";
const char *data_pub = "@shadow/data/update";

const int OLED_SDA = 21;
const int OLED_SCL = 22;
const int LED1_PIN = 4;
const int LED2_PIN = 19;
const int MQ3_PIN = 34;

const uint8_t SCREEN_WIDTH = 128;
const uint8_t SCREEN_HEIGHT = 64;
const int OLED_RESET = -1;
const uint8_t OLED_ADDR = 0x3C;

const unsigned long SENSOR_READ_INTERVAL_MS = 1000; // อ่านเซ็นเซอร์ทุก 1 วินาที
const unsigned long DISPLAY_HOLD_MS = 2500; // แสดงค่าที่วัดได้ 2.5 วินาที ก่อนกลับไปหน้ารอวัดใหม่
const float MIN_REPORT_BAC = 10.0f;
const float BAC_MAX_VALUE = 100.0f;

float threshold1 = 20.0f;
float threshold2 = 50.0f;
float lastBacValue = 0.0f;
float lastDisplayedBac = 0.0f;
int lastRawValue = 0;
int alertLevel = 0;
unsigned long lastSensorReadAt = 0;
unsigned long lastDisplayValueAt = 0;
bool displayReady = false;
bool displayHoldingValue = false;

WiFiClient wifiClient;
PubSubClient mqttClient(wifiClient);
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// จำกัดค่า BAC ให้อยู่ในช่วงที่ระบบรองรับ
float clampBac(float value) {
  if (value < 0.0f) {
    return 0.0f;
  }
  if (value > BAC_MAX_VALUE) {
    return BAC_MAX_VALUE;
```

```
  }
  return value;
}

// ปัดทศนิยมให้เหลือ 1 ตำแหน่งเพื่อแสดงผล/ส่งข้อมูลให้สม่ำเสมอ
float roundOneDecimal(float value) {
  return roundf(value * 10.0f) / 10.0f;
}

// จัดระเบียบค่า threshold ให้อยู่ในช่วงถูกต้องและเรียงจากน้อยไปมาก
void normalizeThresholds() {
  threshold1 = clampBac(threshold1);
  threshold2 = clampBac(threshold2);

  // ถ้าสลับลำดับกันอยู่ ให้สลับกลับเพื่อกันเงื่อนไขผิดพลาด
  if (threshold1 > threshold2) {
    const float temp = threshold1;
    threshold1 = threshold2;
    threshold2 = temp;
  }
}

// อ่านค่าเซ็นเซอร์หลายครั้งแล้วเฉลี่ย เพื่อลดสัญญาณรบกวน
int readAlcoholRaw() {
  long total = 0;
  const int sampleCount = 10;

  for (int sample = 0; sample < sampleCount; sample++) {
    total += analogRead(MQ3_PIN);
    delay(5);
  }

  return total / sampleCount;
}

// แปลงค่าดิบเป็นเปอร์เซ็นต์ ตามสเกล 0-100
float convertRawToBac(int rawValue) {
  return clampBac((rawValue / 4095.0f) * BAC_MAX_VALUE);
}

// วาดข้อมูลสถานะทั้งหมดลง OLED
void renderDisplay(float bacValue, bool showMeasuredValue) {
  // ถ้า OLED ไม่พร้อม ให้ข้ามทันที
  if (!displayReady) {
    return;
  }

  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("Alcohol Monitor");

  display.setTextSize(2);
  display.setCursor(0, 16);
  display.print("Alc: ");
  display.println(roundOneDecimal(showMeasuredValue ? bacValue : 0.0f), 1);

  display.setTextSize(1);
  display.setCursor(0, 42);
  display.print("T1:");
  display.print(roundOneDecimal(threshold1), 0);
  display.print(" T2:");
  display.println(roundOneDecimal(threshold2), 0);
```

```
  display.setCursor(0, 54);
  if (showMeasuredValue) {
    display.print("Sent to cloud");
  } else {
    display.print("Waiting for sample");
  }

  display.display();
}

// อัปเดตระดับแจ้งเตือน และควบคุม LED ตามค่า BAC
void updateAlertOutputs(float bacValue) {
  if (bacValue >= threshold2) {
    alertLevel = 2;
  } else if (bacValue >= threshold1) {
    alertLevel = 1;
  } else {
    alertLevel = 0;
  }

  digitalWrite(LED1_PIN, alertLevel >= 1 ? HIGH : LOW);
  digitalWrite(LED2_PIN, alertLevel >= 2 ? HIGH : LOW);
}

// ส่งสถานะล่าสุดขึ้น Shadow ของ NetPIE
void publishShadow() {
  if (!mqttClient.connected()) {
    Serial.println("Shadow publish skipped: MQTT disconnected");
    return;
  }

  const int led1State = digitalRead(LED1_PIN);
  const int led2State = digitalRead(LED2_PIN);

  StaticJsonDocument<384> doc;
  JsonObject data = doc.createNestedObject("data");
  data["red"] = led2State;
  data["yellow"] = led1State;
  data["led1"] = led1State;
  data["bac"] = roundOneDecimal(lastBacValue);
  data["threshold1"] = roundOneDecimal(threshold1);
  data["threshold2"] = roundOneDecimal(threshold2);

  char payload[384];
  serializeJson(doc, payload, sizeof(payload));
  const bool ok = mqttClient.publish(data_pub, payload, true);
  Serial.print("Shadow publish: ");
  Serial.println(ok ? "OK" : "FAIL");
}

// ส่งข้อมูลการวัดไปยัง topic สถานะและ topic BAC
void publishMeasurement() {
  if (!mqttClient.connected()) {
    Serial.println("MQTT publish skipped: MQTT disconnected");
    return;
  }

  StaticJsonDocument<256> doc;
  doc["bac"] = roundOneDecimal(lastBacValue);
  doc["raw"] = lastRawValue;
  doc["threshold1"] = roundOneDecimal(threshold1);
  doc["threshold2"] = roundOneDecimal(threshold2);
  doc["alertLevel"] = alertLevel;
  doc["millis"] = millis();
```

```
  char payload[256];
  serializeJson(doc, payload, sizeof(payload));
  const bool okStatus = mqttClient.publish(topic_pub, payload);
  const bool okBac = mqttClient.publish(topic_pub_bac, payload);
  Serial.print("MQTT publish status/bac: ");
  Serial.print(okStatus ? "OK" : "FAIL");
  Serial.print(" / ");
  Serial.println(okBac ? "OK" : "FAIL");
}

// ส่งค่าการวัดไปเก็บใน Firebase Realtime Database
void sendToFirebase() {
  // ส่งเฉพาะตอนต่อ WiFi ได้เท่านั้น
  if (WiFi.status() != WL_CONNECTED) {
    return;
  }

  WiFiClientSecure secureClient;
  secureClient.setInsecure();

  HTTPClient https;
  const String url = String(Secrets::FIREBASE_BASE_URL) +
"/measurements.json";
  if (!https.begin(secureClient, url)) {
    Serial.println("Firebase connection failed");
    return;
  }

  https.addHeader("Content-Type", "application/json");

  StaticJsonDocument<256> doc;
  doc["bac"] = roundOneDecimal(lastBacValue);
  doc["raw"] = lastRawValue;
  doc["threshold1"] = roundOneDecimal(threshold1);
  doc["threshold2"] = roundOneDecimal(threshold2);
  doc["alertLevel"] = alertLevel;
  doc["millis"] = millis();

  String body;
  serializeJson(doc, body);
  const int httpCode = https.POST(body);
  Serial.print("Firebase HTTP code: ");
  Serial.println(httpCode);
  https.end();
}

// รับค่า threshold ใหม่จาก JSON และบังคับให้ค่าถูกต้องก่อนใช้งาน
void applyThresholdUpdate(JsonObjectConst source) {
  bool updated = false;

  if (source.containsKey("threshold1")) {
    threshold1 = source["threshold1"].as<float>();
    updated = true;
  }
  if (source.containsKey("threshold2")) {
    threshold2 = source["threshold2"].as<float>();
    updated = true;
  }
  if (source.containsKey("level1")) {
    threshold1 = source["level1"].as<float>();
    updated = true;
  }
  if (source.containsKey("level2")) {
    threshold2 = source["level2"].as<float>();
    updated = true;
  }
```

```
  if (!updated) {
    return;
  }

  normalizeThresholds();
  updateAlertOutputs(lastBacValue);
  publishShadow();
  renderDisplay(lastDisplayedBac, displayHoldingValue);

  Serial.print("Threshold updated: ");
  Serial.print(threshold1);
  Serial.print(", ");
  Serial.println(threshold2);
}

// callback เมื่อมีข้อความ MQTT เข้ามา
void callback(char *topic, byte *payload, unsigned int length) {
  (void)topic;

  String message;
  for (unsigned int index = 0; index < length; index++) {
    message += static_cast<char>(payload[index]);
  }

  Serial.print("Message arrived: ");
  Serial.println(message);

  StaticJsonDocument<256> doc;
  DeserializationError error = deserializeJson(doc, message);
  if (error) {
    Serial.print("Invalid JSON: ");
    Serial.println(error.c_str());
    return;
  }

  JsonObjectConst source = doc.as<JsonObjectConst>();
  if (doc["data"].is<JsonObjectConst>()) {
    source = doc["data"].as<JsonObjectConst>();
  }

  applyThresholdUpdate(source);
}

// เชื่อมต่อ WiFi และรอจนกว่าจะสำเร็จ
void setupWifi() {
  delay(10);
  Serial.print("Connecting to ");
  Serial.println(Secrets::WIFI_SSID);
  WiFi.begin(Secrets::WIFI_SSID, Secrets::WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi connected");
}

// พยายามเชื่อม MQTT ใหม่จนกว่าจะสำเร็จ
void reconnectMQTT() {
  while (!mqttClient.connected()) {
    Serial.println("Attempting MQTT connection...");
    if (mqttClient.connect(
            Secrets::MQTT_CLIENT_ID,
            Secrets::MQTT_USERNAME,
```

```
            Secrets::MQTT_PASSWORD)) {
      Serial.println("Connected to NetPIE");
      mqttClient.subscribe(topic_sub);
      publishShadow();
    } else {
      Serial.print("MQTT failed, rc=");
      Serial.println(mqttClient.state());
      delay(5000);
    }
  }
}

// เริ่มต้นจอ OLED และแสดงหน้าเริ่มต้น
void setupDisplay() {
  Wire.begin(OLED_SDA, OLED_SCL);
  displayReady = display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR);

  if (!displayReady) {
    Serial.println("OLED not found");
    return;
  }

  renderDisplay(0.0f, false);
}

// ตั้งค่าฮาร์ดแวร์และบริการทั้งหมดตอนบูต
void setup() {
  Serial.begin(115200);

  pinMode(LED1_PIN, OUTPUT);
  pinMode(LED2_PIN, OUTPUT);
  pinMode(MQ3_PIN, INPUT);

  digitalWrite(LED1_PIN, LOW);
  digitalWrite(LED2_PIN, LOW);

  analogReadResolution(12);
  analogSetPinAttenuation(MQ3_PIN, ADC_11db);
  normalizeThresholds();

  setupDisplay();
  setupWifi();

  mqttClient.setServer(mqttServer, mqttPort);
  mqttClient.setCallback(callback);
  mqttClient.setBufferSize(512);
}

// วนลูปหลัก: ดูแลการเชื่อมต่อ, อ่านเซ็นเซอร์, อัปเดตแจ้งเตือน, และส่งข้อมูล
void loop() {
  // ถ้าหลุด MQTT ให้เชื่อมใหม่ก่อนทำงานต่อ
  if (!mqttClient.connected()) {
    reconnectMQTT();
  }
  mqttClient.loop();

  const unsigned long now = millis();
  // อ่านค่าเซ็นเซอร์ตามช่วงเวลา ไม่อ่านถี่เกินจำเป็น
  if (now - lastSensorReadAt >= SENSOR_READ_INTERVAL_MS) {
    lastSensorReadAt = now;

    lastRawValue = readAlcoholRaw();
    lastBacValue = roundOneDecimal(convertRawToBac(lastRawValue));
    updateAlertOutputs(lastBacValue);
    publishShadow();
```

```
    Serial.print("BAC: ");
    Serial.println(lastBacValue);

    // ส่งขึ้น cloud เฉพาะค่าที่เกินเกณฑ์ขั้นต่ำ
    if (lastBacValue > MIN_REPORT_BAC) {
      lastDisplayedBac = lastBacValue;
      lastDisplayValueAt = now;
      displayHoldingValue = true;
      renderDisplay(lastDisplayedBac, true);
      publishMeasurement();
      sendToFirebase();
    }
  }

  // ครบเวลาค้างหน้าจอแล้วกลับไปสถานะรอวัด
  if (displayHoldingValue && (now - lastDisplayValueAt >= DISPLAY_HOLD_MS)) {
    displayHoldingValue = false;
    lastDisplayedBac = 0.0f;
    renderDisplay(0.0f, false);
  }
}
```